Laporan Tugas Besar 1 IF3270 Pembelajaran Mesin

Feedforward Neural Network

Muhammad Dicky Isra
13523075
13523075@std.stei.itb.ac.id

Guntara Hambali
13523114
13523114@std.stei.itb.ac.id

Reza Ahmad Syarif
13523119
13523119@std.stei.itb.ac.id

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

2026

Daftar Isi

Bab I: Deskripsi Persoalan **2**
Bab II: Pembahasan **3**
Deskripsi Kelas Sequential 3
Deskripsi Kelas Dense 4
Fungsi Aktivasi 5
Fungsi Loss 7
Forward Propagation 9
Backward Propagation dan Weight Update 10
Bab III: Hasil Eksperimen dan Analisis **12**
Pengaruh Depth dan Width 12
Pengaruh Fungsi Aktivasi Hidden Layer 14
Pengaruh Learning Rate 17
Pengaruh Inisialisasi Bobot 19
Pengaruh Regularisasi 21
Pengaruh Optimizer 23
Pengaruh RMS Normalizer 25
Perbandingan dengan Library Sklearn 27
Bab IV: Kesimpulan dan Saran **28**
Kesimpulan 28
Saran 28
Pembagian Tugas **30**
Lampiran **30**
Daftar Pustaka **30**

Bab I: Deskripsi Persoalan

Tugas Besar 1 mata kuliah IF3270 Pembelajaran Mesinl menugaskan mahasiswa untuk mengimplementasikan Feedforward Neural Network (FFNN) dari nol (*from scratch*) tanpa menggunakan library deep learning yang sudah ada, seperti TensorFlow atau PyTorch. Persoalan ini menuntut pemahaman mendalam tentang mekanisme jaringan saraf tiruan (*Artificial Neural Network*), mulai dari *forward propagation*, perhitungan loss function, hingga pembaruan bobot dengan *backward propagation* menggunakan algoritma gradient descent.

Setiap *layer* jaringan memiliki atribut berupa jumlah neuron, fungsi aktivasi, serta bobot dan bias yang perlu diinisialisasi, disimpan, dan diperbarui selama proses pelatihan. Implementasi mendukung berbagai fungsi aktivasi, seperti Linear, ReLU, Sigmoid, Tanh, dan Softmax. Selain itu, juga diimplementasikan fungsi loss MSE, Binary Cross-Entropy, dan Categorical Cross-Entropy. Serta metode inisialisasi bobot zero, uniform, dan normal. Selain itu, model juga mendukung regularisasi L1 dan L2 untuk mencegah overfitting, serta bisa memproses data dalam bentuk batch.

Bab II: Pembahasan

Deskripsi Kelas Sequential

Kelas Sequential digunakan sebagai representasi model FFNN secara keseluruhan. Kelas ini menyimpan daftar layer yang disusun secara berurutan, konfigurasi loss function, serta optimizer yang dipakai dalam proses training.

```
class Sequential:
  def __init__(self) -> None:
      self._layers: list[Dense] = []
      self._optimizer = None

   def add(self, layer: Dense) -> None:
      if len(self._layers) == 0 and not layer.is_first_layer():
         raise ValueError("First layer must have input dimension specified")
      if len(self._layers) > 0 and layer.is_first_layer():
         raise ValueError("Subsequent layer must not have input dimension specified")
      if len(self._layers) > 0:
         previous_layer = self._layers[-1]
         layer.build(previous_layer.output_dim())
      self._layers.append(layer)

  def compile(
      self,
      loss: Literal["mse", "binary_crossentropy", "categorical_crossentropy"],
      optimizer = SGD(0.001)
   ) -> None:
      self._loss = loss
      self._optimizer = optimizer
```

Methode add digunakan untuk menambahkan layer ke dalam model. Layer pertama wajib memiliki input_dim, sedangkan layer-layer berikutnya akan mendapat dimensi input secara otomatis dari output layer sebelumnya. Hal ini dilakukan sehingga, model dapat dibangun secara tanpa perlu menyatakan ulang ukuran input di setiap layer.

Method compile digunakan untuk menyimpan loss function yang akan dipakai mode serta optimizer yang bertugas melakukan update parameter. Pada program ini, loss yang dibuat diantara lain mse, binary_crossentropy, dan categorical_crossentropy.

Method forward melakukan forward propagation dengan meneruskan output suatu layer menjadi input bagi layer berikutnya hingga menghasilkan prediksi akhir. Method backward digunakan untuk menghitung gradien pada seluruh layer secara berurutan dari layer output menuju layer awal, lalu menyerahkan pembaruan parameter kepada optimizer. Method fit

mengatur loop pelatihan selama beberapa epoch, membagi data ke dalam batch, menghitung training loss dan validation loss, lalu menyimpan histori pelatihan. Selain itu, kelas ini juga memiliki method save dan load untuk menyimpan dan memuat model yang telah dibuat sebelumnya.

Deskripsi Kelas Dense

Kelas Dense merepresentasikan satu *fully connected layer* pada FFNN. Setiap objek Dense menyimpan jumlah neuron keluaran, dimensi input, jenis fungsi aktivasi, bobot, bias, *cache* hasil komputasi forward, serta gradien terhadap bobot dan bias.

```
class Dense:
    def __init__(
        self,
        n_neuron: int,
        activation: ActivationName,
        kernel_regularizer=None,
        input_dim: Optional[int] = None,
        weight_initializer: WeightInitializer = "random_normal",
        seed: Optional[int] = None,
        lower_bound: float = -0.5,
        upper_bound: float = 0.5,
        mean: float = 0.0,
        variance: float = 1.0,
    ) -> None:
        self._n_neuron = n_neuron
        self._input_dim = input_dim
        self._activation = activation
        self._weight_initializer = weight_initializer
        self._kernel_regularizer = kernel_regularizer
        self.dW = None
        self.dB = None
```

Pada saat layer dibuat, bobot dan bias diinisialisasi menggunakan salah satu metode, yaitu zeros, random_uniform, atau random_normal. Untuk inisialisasi uniform, parameter yang digunakan adalah lower_bound, upper_bound, dan seed. Untuk inisialisasi normal, parameter yang digunakan adalah mean, variance, dan seed. Seed disini ditujukan agar model dapat diproduksi kembali.

Method forward pada layer dense menghitung transformasi linear:

$$Z = XW + b$$

Selanjutnya menerapkan fungsi aktivasi pada hasil tersebut untuk memperoleh output layer:

```
A = f(Z)
```

Selama proses forward, layer juga menyimpan input batch, nilai pre-activation, dan output aktivasi sebagai *cache* yang dibutuhkan pada backward propagation.

Method backward menghitung error term untuk layer output dan hidden layer, lalu menghitung gradien bobot dan gradien bias. Pada program ini, Dense.backward hanya bertugas menghitung gradien, sedangkan pembaruan parameter akan dilakukan secara terpisah oleh optimizer. Jika layer menggunakan regularisasi, kontribusi gradien regularisasi juga ditambahkan ke gradien bobot..

Fungsi Aktivasi

Program ini mendukung lima fungsi aktivasi, sebagai berikut:

1. Linear
2. ReLU
3. Sigmoid
4. Tanh
5. Softmax
6. Swish
7. Leaky ReLU

Setiap fungsi aktivasi diimplementasikan bersama turunannya agar dapat digunakan baik pada tahap forward propagation maupun backward propagation. Untuk fungsi softmax, implementasi menggunakan teknik stabilisasi numerik dengan mengurangkan nilai maksimum setiap baris sebelum eksponensial dihitung. Hal ini bertujuan untuk mengurangi risiko *overflow* saat menangani nilai input yang besar.

```
def linear(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return x

def linear_derivative(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return np.ones_like(x)

def relu(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return np.maximum(0.0, x)

def relu_derivative(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return (x > 0).astype(np.float64)

def sigmoid(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return 1.0 / (1.0 + np.exp(-x))

def sigmoid_derivative(x: NDArray[np.float64]) -> NDArray[np.float64]:
```

```
    s = sigmoid(x)
    return s * (1 - s)

def tanh(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return np.tanh(x)

def tanh_derivative(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return 1.0 - np.tanh(x) ** 2

def softmax(x: NDArray[np.float64]) -> NDArray[np.float64]:
    shifted = x - np.max(x, axis=1, keepdims=True)
    exp_x = np.exp(shifted)
    return exp_x / np.sum(exp_x, axis=1, keepdims=True)

def softmax_derivative(x: NDArray[np.float64]) -> NDArray[np.float64]:
    s = softmax(x)
    diag_s = s[:, :, np.newaxis] * np.eye(s.shape[1])
    outer_s = s[:, np.newaxis, :] * s[:, :, np.newaxis]
    jacobian = diag_s - outer_s
    return jacobian

def swish(x: NDArray[np.float64]) -> NDArray[np.float64]:
    return x * sigmoid(x)

def swish_derivative(x: NDArray[np.float64]) -> NDArray[np.float64]:
    s = sigmoid(x)
    return s + x * s * (1 - s)

def leaky_relu(x: NDArray[np.float64], alpha: float = 0.01) -> NDArray[np.float64]:
    return np.where(x > 0, x, alpha * x)

def leaky_relu_derivative(x: NDArray[np.float64], alpha: float = 0.01) -> NDArray[np.float64]:
    return np.where(x > 0, 1.0, alpha)

def apply_activation(
    name: ActivationName, x: NDArray[np.float64]
) -> NDArray[np.float64]:
    if name == "linear":
        return linear(x)
    if name == "relu":
        return relu(x)
    if name == "sigmoid":
        return sigmoid(x)
    if name == "tanh":
        return tanh(x)
    if name == "softmax":
        return softmax(x)
    if name == "swish":
        return swish(x)
    if name == "leaky_relu":
```

```
        return leaky_relu(x)
    raise ValueError(f"Unsupported activation: {name}")

def apply_activation_derivative(
    name: ActivationName, x: NDArray[np.float64]
) -> NDArray[np.float64]:
    if name == "linear":
        return linear_derivative(x)
    if name == "relu":
        return relu_derivative(x)
    if name == "sigmoid":
        return sigmoid_derivative(x)
    if name == "tanh":
        return tanh_derivative(x)
    if name == "softmax":
        return softmax_derivative(x)
    if name == "swish":
        return swish_derivative(x)
    if name == "leaky_relu":
        return leaky_relu_derivative(x)
    raise ValueError(f"Unsupported activation: {name}")
```

Kode diatas merupakan implementasi dari fungsi aktivasi Linear, ReLU, Sigmoid, Tanh, Softmax, Swish, dan Leaky ReLU yang akan digunakan pada proses forward propagation. Selain itu juga didefinisikan turunannya yang nantinya akan digunakan dalam proses backward propagation.

Fungsi Loss

Program ini mendukung tiga fungsi loss, sebagai berikut:

1. Mean Squared Error (MSE)
2. Binary Cross-Entropy
3. Categorical Cross-Entropy

Implementasi loss cross-entropy menggunakan clipping kecil (epsilon) pada nilai prediksi untuk mencegah terjadinya log(0) yang dapat menyebabkan hasil perhitungan menjadi tidak stabil secara numerik. Selain fungsi loss, turunan pertama masing-masing loss juga diimplementasikan agar dapat langsung digunakan dalam proses backward propagation.

Forward Propagation

Forward propagation merupakan proses awal pada Feedforward Neural Network (FFNN) untuk menghasilkan prediksi dari suatu input. Pada tahap ini, masukan akan diproses secara

berurutan dari layer pertama hingga layer terakhir dengan menggunakan bobot (*weights*), bias, serta fungsi aktivasi yang dimiliki oleh setiap neuron di jaringan. Input dapat diberikan dalam bentuk batch sehingga dalam satu kali pemanggilan forward dapat diproses lebih dari satu sampel sekaligus.

```
class Sequential:
   def forward(self, inputs: NDArray[np.float64]) -> NDArray[np.float64]:
        output = np.asarray(inputs, dtype=np.float64)
        for layer in self._layers:
            output = layer.forward(output)
        return output
class Dense:


   def forward(self, inputs: NDArray[np.float64]) -> NDArray[np.float64]:
        if self._weights is None or self._bias is None:
            raise ValueError("Layer weights are not initialized. Call build()
first.")


        batch_inputs = np.asarray(inputs, dtype=np.float64)
        if batch_inputs.ndim == 1:
            batch_inputs = batch_inputs.reshape(1, -1)


        if batch_inputs.shape[1] != self._input_dim:
            raise ValueError(
                f"Expected input with {self._input_dim} features, got
{batch_inputs.shape[1]}"
            )


        self._last_input = batch_inputs
        self._last_linear_output = batch_inputs @ self._weights + self._bias
        self._last_output = apply_activation(self._activation,
self._last_linear_output)
        return self._last_output
```

Secara umum, alur dari forward propagation pada model adalah sebagai berikut:

1. Input batch diterima oleh layer pertama.
2. Setiap layer menghitung kombinasi linear antara input, bobot, dan bias.
3. Hasil kombinasi linear dilewatkan ke fungsi aktivasi layer.
4. Output layer saat ini diteruskan ke layer berikutnya.
5. Setelah layer terakhir selesai, model menghasilkan prediksi akhir.

Backward Propagation dan Weight Update

Backward propagation dipakai untuk menghitung gradien loss terhadap bobot dan bias pada setiap layer. Proses ini dilakukan dari layer output menuju layer input dengan menggunakan chain rule.

Pada level model, method backward awalnya menghitung error term pada output layer. Setelah itu, model melakukan iterasi mundur (*backward*) ke semua hidden layer. Setelah semua gradien diperoleh, optimizer akan memperbarui parameter tiap layer.

```
class Sequential:
   def backward(self, target: NDArray[np.float64], outputs:
NDArray[np.float64], batch_size: int):


        layer_size = len(self._layers)
        last_layer = self._layers[-1]
        error_terms = last_layer.backward(predictions=outputs,target=target,
loss=self._loss, batch_size=batch_size)
        for i in range(layer_size-2, -1, -1):
            prev_layer_weights = self._layers[i+1].weights
            error_terms =
self._layers[i].backward(prev_error_terms=error_terms,
prev_layer_weights=prev_layer_weights, loss=self._loss,
batch_size=batch_size)
        for layer in self._layers:
            self._optimizer.update(layer, layer.dW, layer.dB)
```

Pada level layer, terdapat dua kasus. Untuk output layer, error term dihitung dari turunan loss terhadap prediksi. Untuk hidden layer, error term dihitung dari error term layer berikutnya dan bobot layer berikutnya. Setelah error term didapatkan, gradien bobot dan bias dihitung dengan menggunakan *cache* input batch.

```
class Dense:
   def backward(
        self,
        loss,
        target: Optional[NDArray[np.float64]] = None,
        predictions: Optional[NDArray[np.float64]] = None,
        prev_layer_weights: Optional[NDArray[np.float64]] = None,
        prev_error_terms: Optional[NDArray[np.float64]] = None,
        batch_size: int = 1,
    ) -> NDArray[np.float64]:
        if target is not None:
            error_terms = self._compute_output_error_terms(
                loss, target, predictions, batch_size
            )
        else:
            error_terms = self._compute_hidden_error_terms(
                prev_error_terms, prev_layer_weights, batch_size
            )


        self.dW = self._last_input.T @ error_terms


        if self._kernel_regularizer is not None:
            self.dW += self._kernel_regularizer.gradient(self._weights)
```

```
        self.dB = np.sum(error_terms, axis=0, keepdims=True)

        return error_terms
```

Untuk kasus output layer dengan aktivasi softmax dan loss categorical_crossentropy, implementasi menggunakan bentuk predicted - target.

Setelah gradien dihitung, pembaruan parameter dilakukan oleh optimizer. Jika optimizer yang digunakan adalah SGD, maka bentuk pembaruannya adalah:

```
W := W - lr * dW
b := b - lr * db
```

Implementasi update parameter pada optimizer adalah sebagai berikut.

```
class SGD(Optimizer):
    def __init__(self, lr=0.01):
        self.lr = lr


    def update(self, layer, dW, dB):
        layer._weights -= self.lr * dW
        layer._bias -= self.lr * dB
```

Dengan demikian, Dense.backward hanya bertugas untuk menghitung gradien, sedangkan optimizer bertugas memperbarui bobot dan bias. Dengan cara ini, arsitektur program lebih modular dan memudahkan penggantian metode update, misalnya dari SGD ke Adam, tanpa perlu mengubah logika backward propagation.

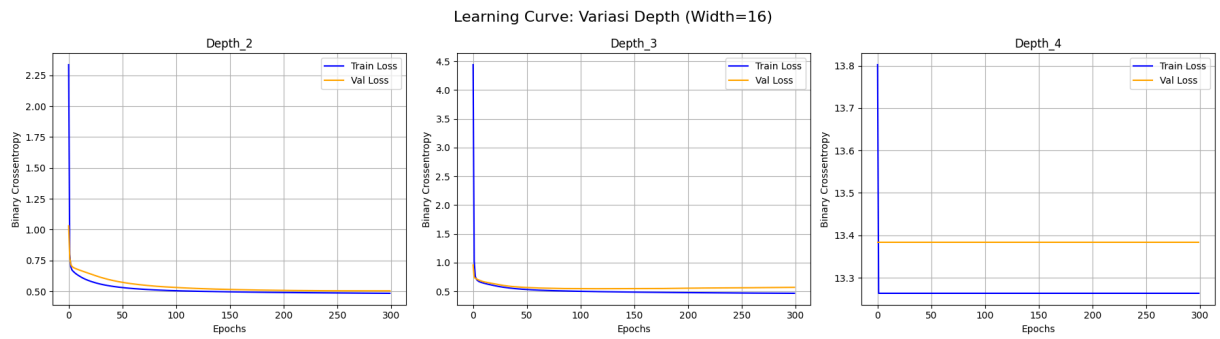
Bab III: Hasil Eksperimen dan Analisis

Pengaruh Depth dan Width

Berikut merupakan hasil eksperimen pengaruh depth (banyak layer) dan width (banyak neuron per layer).

Kombinasi Depth (Width Tetap 16 Neuron)

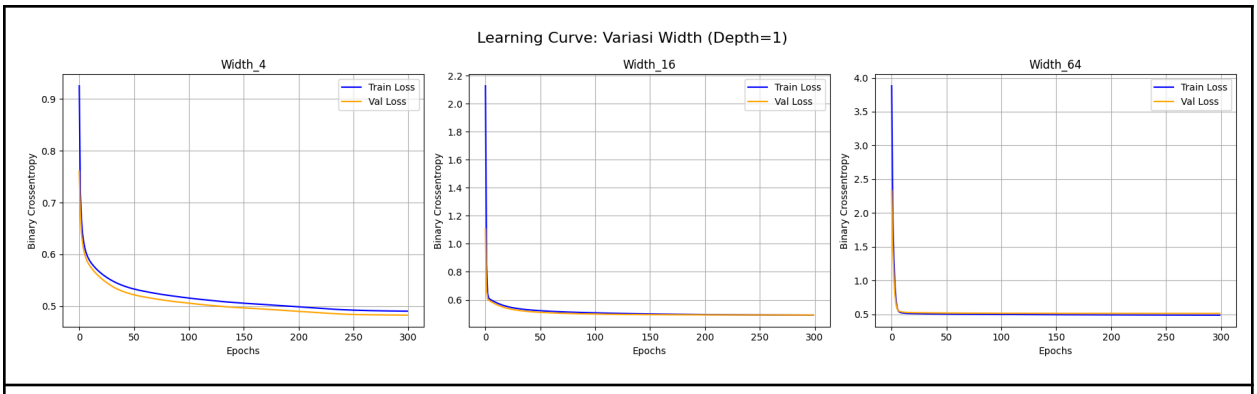
```
Eksperimen Variasi Depth
Training Depth_2 (2 Layer, 16 Neuron)
Training Depth_3 (3 Layer, 16 Neuron)
Training Depth_4 (4 Layer, 16 Neuron)
```




```
Evaluasi Prediksi: Variasi Depth

[Depth_2]:
- Accuracy  : 0.7525
- Precision : 0.7626
- Recall    : 0.8653
- F1-Score  : 0.8107

[Depth_3]:
- Accuracy  : 0.7390
- Precision : 0.7513
- Recall    : 0.8580
- F1-Score  : 0.8011

[Depth_4]:
- Accuracy  : 0.6125
- Precision : 0.6125
- Recall    : 1.0000
- F1-Score  : 0.7597
```

Kombinasi Width (Depth Tetap 1 Hidden Layer)

```
Eksperimen Variasi Width
Training Width_4 (1 Layer, 4 Neuron)
Training Width_16 (1 Layer, 16 Neuron)
Training Width_64 (1 Layer, 64 Neuron)
```




```
Evaluasi Prediksi: Variasi Width

[Width_4]:
- Accuracy  : 0.7625
- Precision : 0.7741
- Recall    : 0.8645
- F1-Score  : 0.8168

[Width_16]:
- Accuracy  : 0.7570
- Precision : 0.7622
- Recall    : 0.8767
- F1-Score  : 0.8155

[Width_64]:
- Accuracy  : 0.7510
- Precision : 0.7461
- Recall    : 0.8996
- F1-Score  : 0.8157
```

Berdasarkan hasil eksperimen variasi *width,* penambahan jumlah neuron pada satu hidden layer memberikan dampak yang signifikan terhadap kecepatan konvergensi model. Dapat dilihat bahwa kurva *loss* 64 neuron menukik jauh lebih tajam daripada kurva lainnya. Meskipun belajar lebih cepat, metrik akhir ketiga variasi tersebut tetap stabil dengan nilai akurasi sekitar 75-76% dan F1-Score sekitar 81%. Hal ini membuktikan bahwa 4 neuron sebenarnya sudah cukup untuk merepresentasikan kompleksitas dataset yang digunakan. Penambahan neuron lebih banyak hanya berfungsi menyediakan jalur komputasi paralel tambahan tanpa memicu overfitting.

Sebaliknya, variasi depth justur menunjukkan penurunan performa seiring bertambahnya kedalaman. Depth 2 merupakan arsitektur paling optimal dengan akurasi 75,25%. Pada Depth 4 terjadi kegagalan fatal yang ditandai dengan kurva *loss* yang langsung mendatar di angka tinggi sejak *epoch* pertama. Secara metrik, Depth 4 menghasilkan *Recall* 1.0000 dengan *Accuracy* dan *Precision* yang identik yaitu 0.6125. Kegagalan pada Depth 4 ini disebabkan oleh masalah *vanishing gradient*. Model harus mempropagasi balik gradien melewati banyak lapisan hanya dengan mengandalkan algoritma *Stochastic Gradient Descent,* sinyal gradien melemah dan gagal mencapai lapisan awal. Akibatnya, model gagal belajar dan mengambil jalan pintas dengan menebak kelas mayoritas yaitu Kelas 1 untuk seluruh data uji.

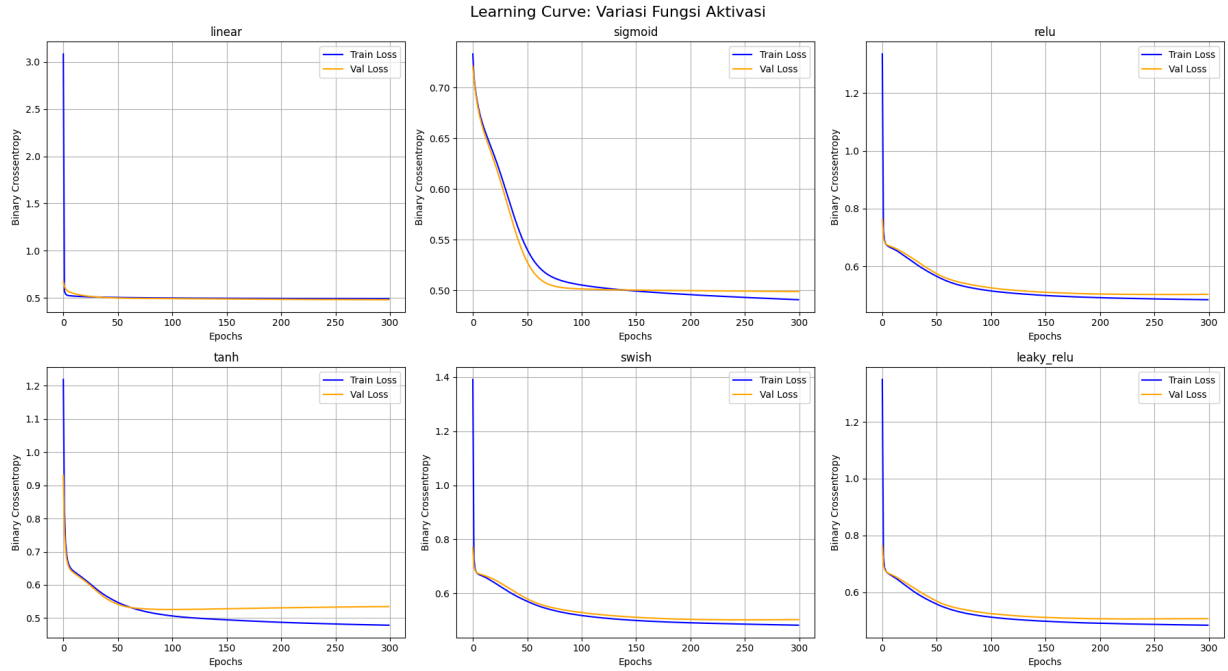
Kesimpulannya adalah depth yang lebih besar pada model tidak selalu menjamin kinerja yang lebih bagus jika algoritma pembelajarannya terbatas. Untuk dataset ini, arsitektur yang dangkal namun lebar terbukti jauh lebih stabil dibandingkan jaringan yang sangat dalam.

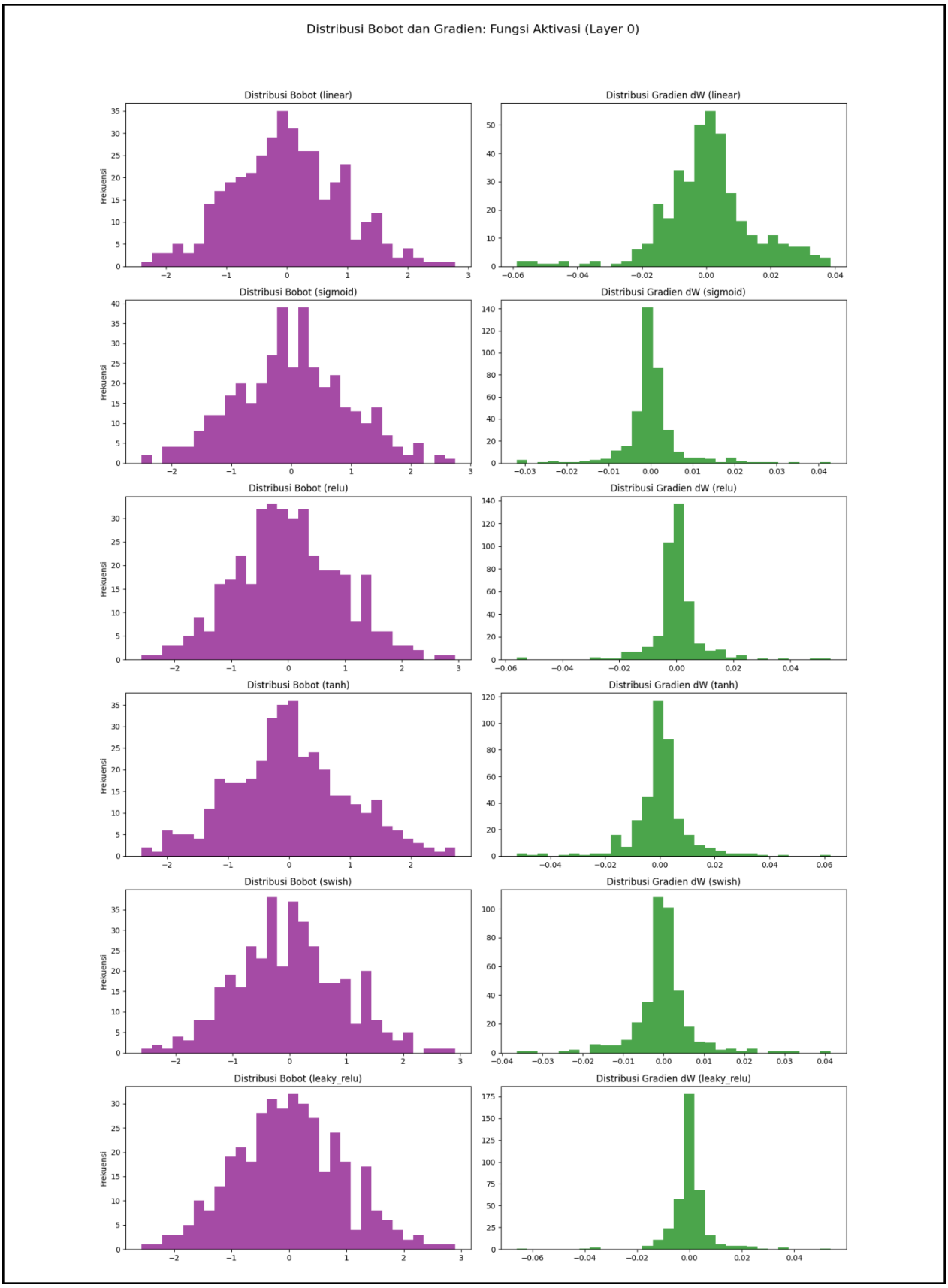
Pengaruh Fungsi Aktivasi Hidden Layer

Berikut merupakan hasil eksperimen pengaruh fungsi aktivasi pada hidden layer.

Pengaruh Fungsi Aktivasi

```
Eksperimen Pengaruh Fungsi Aktivasi Hidden Layer
Training dengan aktivasi: LINEAR
Training dengan aktivasi: SIGMOID
Training dengan aktivasi: RELU
Training dengan aktivasi: TANH
Training dengan aktivasi: SWISH
Training dengan aktivasi: LEAKY_RELU
```




Distribusi Bobot dan Gradien: Fungsi Aktivasi (Layer 0)
Distribusi Bobot (linear)
Distribusi Gradien dW (linear)
Distribusi Bobot (sigmoid)
Distribusi Gradien dW (sigmoid)
Distribusi Bobot (relu)
Distribusi Gradien dW (relu)
Distribusi Bobot (tanh)
Distribusi Gradien dW (tanh)
Distribusi Bobot (swish)
Distribusi Gradien dW (swish)
Distribusi Bobot (leaky_relu)
Distribusi Gradien dW (leaky_relu)
Frekuensi

```
Evaluasi Prediksi: FUngsi Aktivasi

[linear]:
- Accuracy  : 0.7685
- Precision : 0.7729
- Recall    : 0.8808
- F1-Score  : 0.8233

[sigmoid]:
- Accuracy  : 0.7635
- Precision : 0.7793
- Recall    : 0.8563
- F1-Score  : 0.8160

[relu]:
- Accuracy  : 0.7605
- Precision : 0.7687
- Recall    : 0.8710
- F1-Score  : 0.8167

[tanh]:
- Accuracy  : 0.7440
- Precision : 0.7635
- Recall    : 0.8433
- F1-Score  : 0.8014

[swish]:
- Accuracy  : 0.7530
- Precision : 0.7613
- Recall    : 0.8694
- F1-Score  : 0.8117

[leaky_relu]:
- Accuracy  : 0.7560
- Precision : 0.7638
- Recall    : 0.8710
- F1-Score  : 0.8139
```

Berdasarkan hasil eksperimen variasi fungsi aktivasi, penggunaan fungsi aktivasi linear, relu, serta tambahan fungsi yaitu swish dan leaky_relu menghasilakn konvergensi *loss* yang sangat cepat dan menukik tajam sejak awal *epoch*. Sebaliknya, kurva pembelajaran pada fungsi aktivasi sigmoid sempat mengalami fase stagnan di awal sebelum akhirnya turun secara drastis pada pertengahan pelatihan. Di akhir *epoch*, hampir seluruh model menunjukkan pergerakan *validation loss* yang stabil dan berhimpit dengan *training loss*. Kecuali pada fungsi aktivasi tanh yang ujungnya mulai memisah yang menandakan terjadinya *overfitting* ringan.

Meskipun *learning curve* masing-masing fungsi aktivasi berbeda, distribusi matriks bobot akhir pada keenam fungsi tersebut relatif seragam dengan bentuk kurva normal berada pada rentang -2 hingga 3. Pada histogram distribusi gradien, terlihat bahwa keenam fungsi aktivasi memiliki sebaran gradien yang sangat sempit dan terpusat memuncak pada angka nol.

Berdasarkan hasil metrik prediksi akhir yang diperoleh, keenam fungsi aktivasi cukup bersaing. Pada pengujian ini, aktivasi linear unggul dengan tingkat akurasi sebesar 76,85% dan F1-Score 0,8233. Nilai metrik yang tinggi pada linear ini menandakan bahwa distribusi fitur pada dataset ini pada dasarnya cukup mudah dipisahkan secara linear (linearly separable). Di sisi lain, fungsi tanh menghasilkan performa terendah dengan akurasi 74,40% yang sejalan dengan tren *overfitting* yang terlihat pada kurva *loss.*

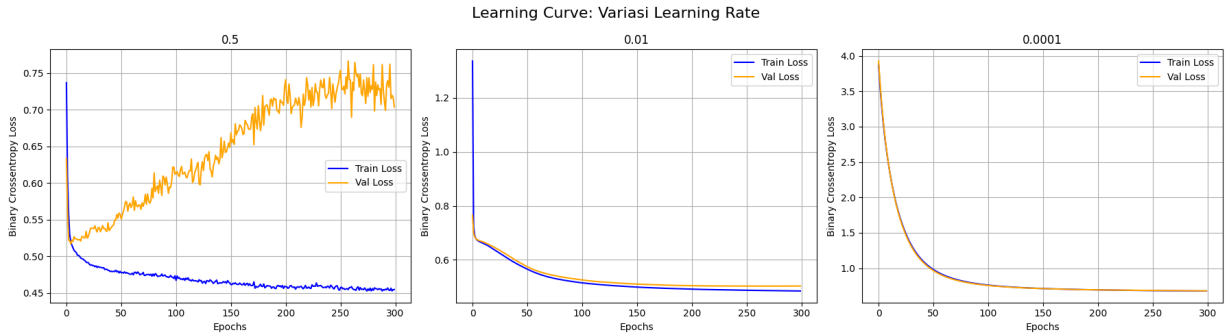
Kesimpulannya adalah model dengan fungsi aktivasi linear atau sigmoid mampu memberikan akurasi terbaik. Sementara itu, variasi dari ReLU yaitu relu, swish, dan leaky_relu, dapat menjadi alternatif arsitektur yang tangguh dan stabil pada dataset ini.

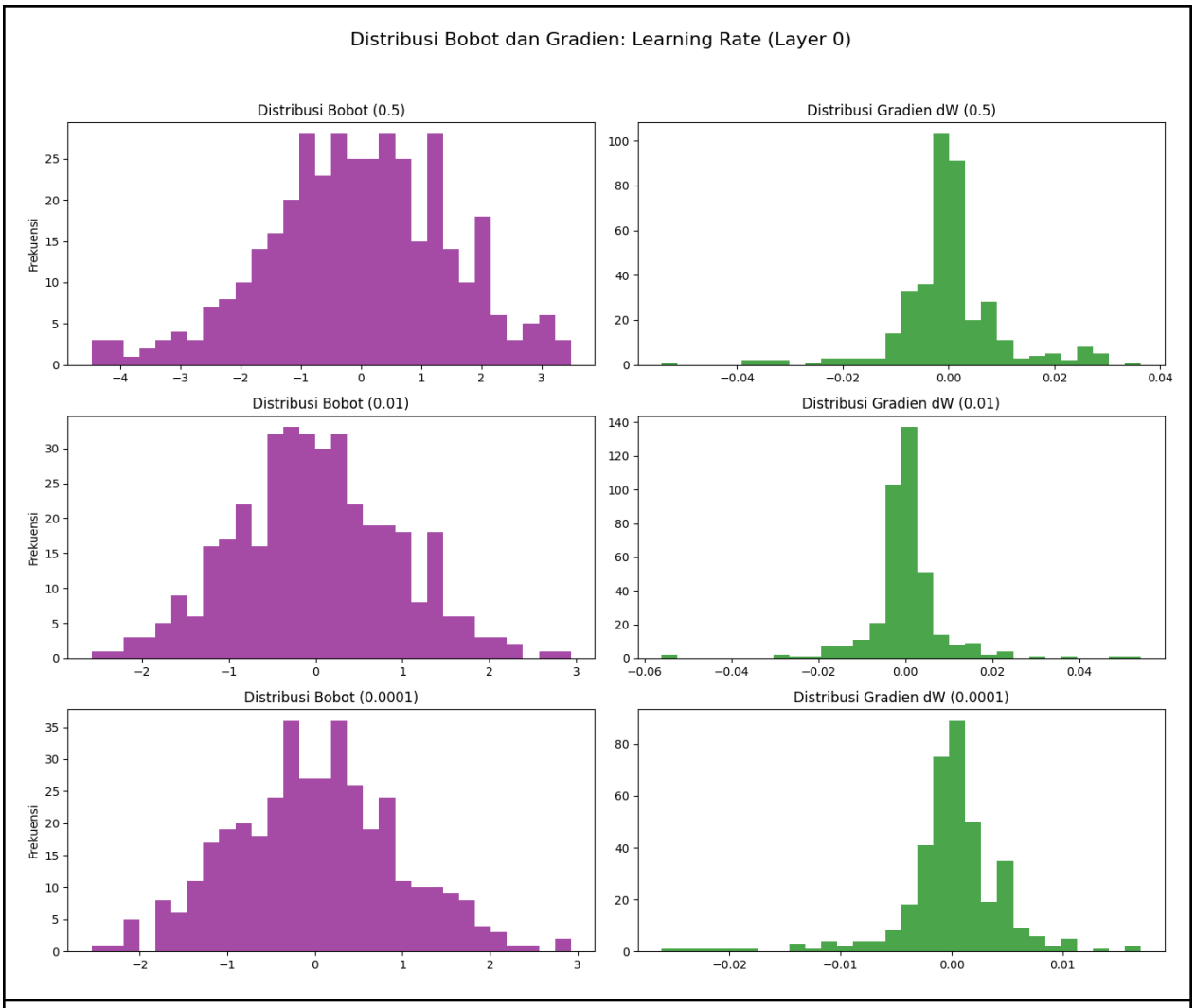
Pengaruh Learning Rate

Berikut merupakan hasil eksperimen pengaruh variasi *learning rate*.

Pengaruh Learning Rate

`Eksperimen Pengaruh Learning Rate`
`Training dengan Learning Rate: 0.5`
`Training dengan Learning Rate: 0.01`
`Training dengan Learning Rate: 0.0001`






```
Evaluasi Prediksi: Learning Rate

[0.5]:
- Accuracy  : 0.7130
- Precision : 0.7033
- Recall    : 0.9192
- F1-Score  : 0.7969

[0.01]:
- Accuracy  : 0.7605
- Precision : 0.7687
- Recall    : 0.8710
- F1-Score  : 0.8167

[0.0001]:
- Accuracy  : 0.6025
- Precision : 0.6155
- Recall    : 0.9355
- F1-Score  : 0.7425
```

Berdasarkan eksperimen learning rate, terdapat hasil yang cukup mencolok. Terlihat bahwa learning rate yang besar (diwakili dengan lr=0.5) memiliki learning curve yang sangat buruk pada dataset validasi. Terlihat bahwa memang pada awalnya validation loss menurun. Akan

tetapi, setelah itu, validation loss terlihat menaik dengan tren yang sangat konsisten, meskipun terdapat gejolak/riak yang sangat besar. Hal ini disebabkan oleh learning rate yang besar akan membuat weight melompat secara drastis bahkan ketika sudah dekat dengan minimumglobal. Akibatnya, model sulit untuk konvergen. Hasil evaluasi menunjukkan akurasi 71,3% dan F1-Score 0,7969.

Sementara itu, pada learning rate yang kecil (diwakli lr=0.0001), terlihat bahwa kurva pembelajaran mampu konvergen/melandai walaupun lambat dan loss yang dihasilkan masih agak besar. Akibat nilai loss yang tidak terlalu memuaskan, hasil evaluasi juga tidak memuaskan dengan nilai akurasi aja 60,25% dan f1-score 0,7425. Ini terjadi karena learning rate yang terlalu kecil menyebabkan grafik loss tidak dapat keluar dari minimum lokal untuk menuju minimim global

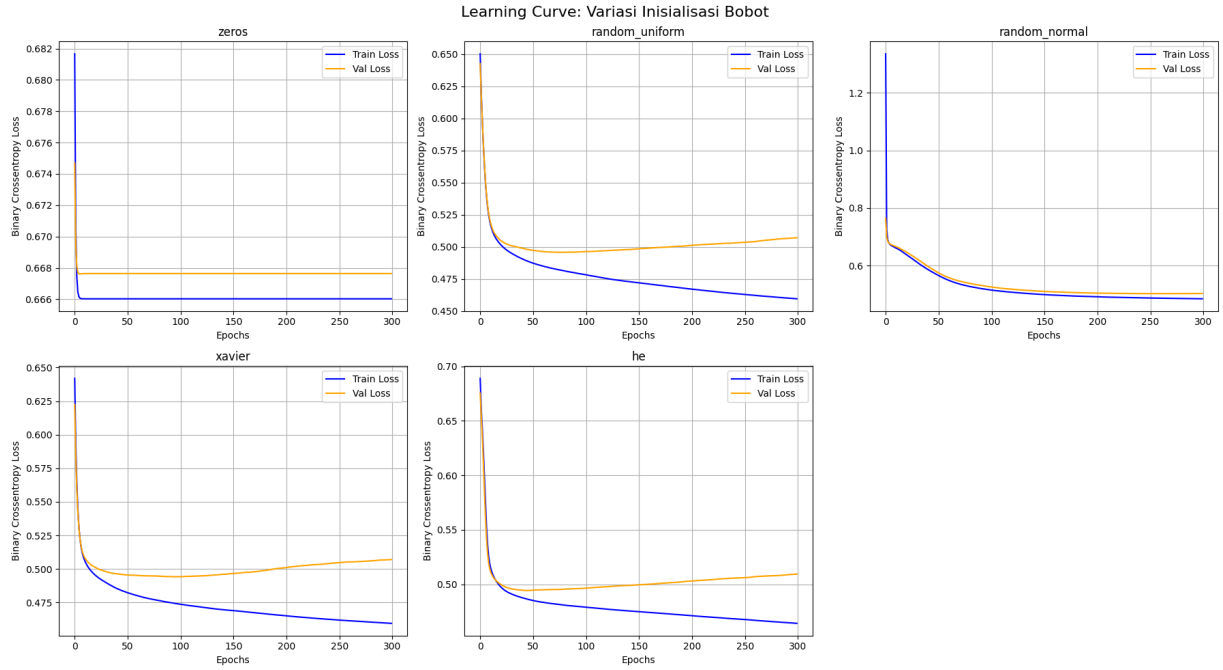
Terakhir, pada learning rate yang sedang (diwakili lr=0.01) tampak bahwa kurva pembelajaran menurun cukup drastis dan akhirnya konvergen pada nilai loss yang cukup memuaskan. Terlihat juga bahwa hasil evaluasinya paling maksimal di antara learning rate lain, dengan akurasi 76,05% dan F1-Score 0,8167. Ini terjadi karena model dapat menyeimbangkan antara dapat keluar dari minimum lokal, dengan kemampuan untuk dapat konvergen tanpa gejolak.

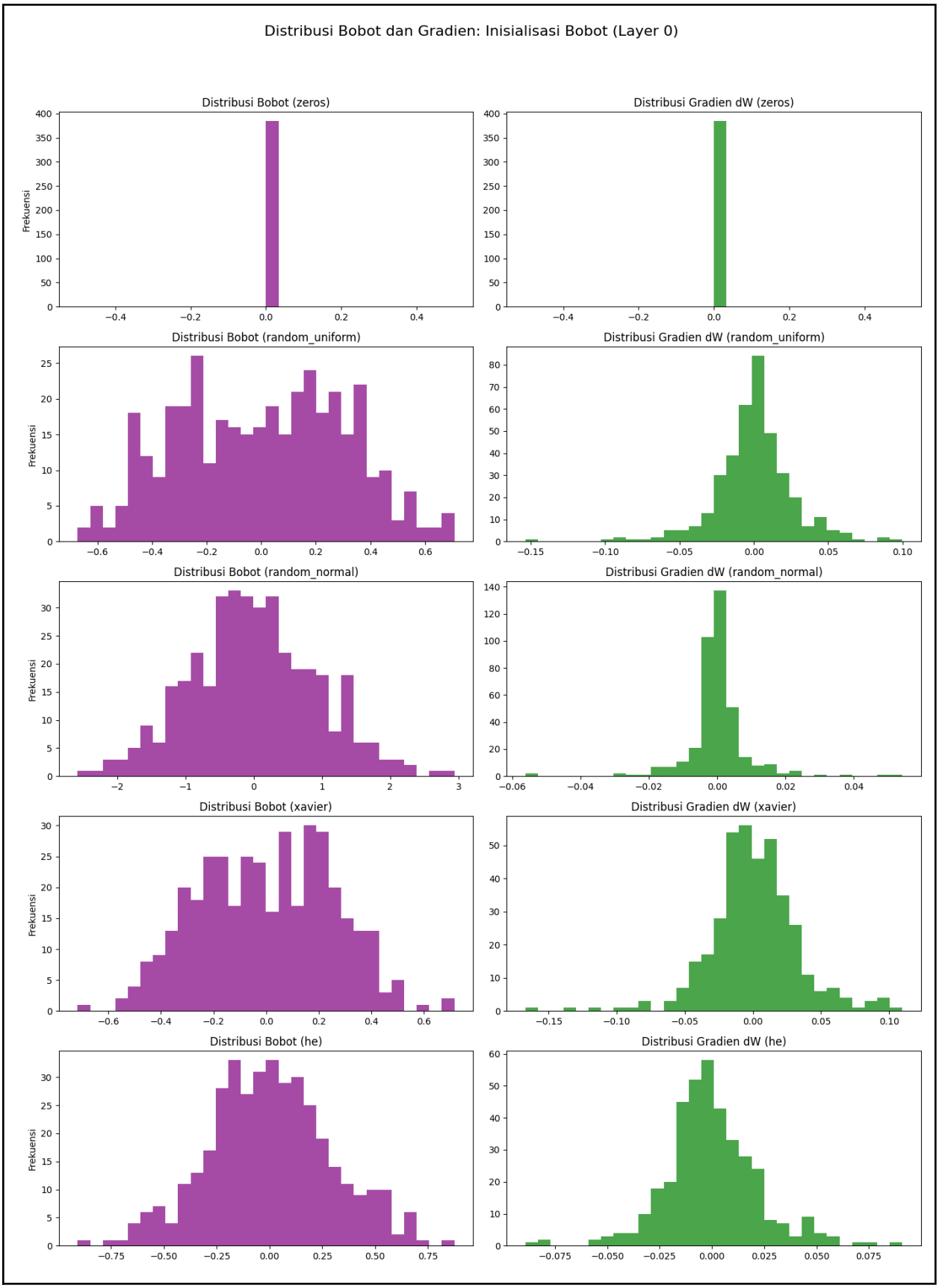
Pengaruh Inisialisasi Bobot

Berikut merupakan hasil eksperimen pengaruh inisialisasi bobot.

Pengaruh Inisialisasi Bobot

```
Eksperimen Pengaruh Inisialisasi Bobot
Training dengan Inisialisasi: zeros
Training dengan Inisialisasi: random_uniform
Training dengan Inisialisasi: random_normal
Training dengan Inisialisasi: xavier
Training dengan Inisialisasi: he
```




Distribusi Bobot dan Gradien: Inisialisasi Bobot (Layer 0)
Distribusi Bobot (zeros)
Distribusi Gradien dW (zeros)
Distribusi Bobot (random_uniform)
Distribusi Gradien dW (random_uniform)
Distribusi Bobot (random_normal)
Distribusi Gradien dW (random_normal)
Distribusi Bobot (xavier)
Distribusi Gradien dW (xavier)
Distribusi Bobot (he)
Distribusi Gradien dW (he)
Frekuensi

```
Evaluasi Prediksi: Inisialisasi Bobot

[zeros]:
- Accuracy  : 0.6125
- Precision : 0.6125
- Recall    : 1.0000
- F1-Score  : 0.7597

[random_uniform]:
- Accuracy  : 0.7415
- Precision : 0.7634
- Recall    : 0.8376
- F1-Score  : 0.7988

[random_normal]:
- Accuracy  : 0.7605
- Precision : 0.7687
- Recall    : 0.8710
- F1-Score  : 0.8167

[xavier]:
- Accuracy  : 0.7600
- Precision : 0.7761
- Recall    : 0.8547
- F1-Score  : 0.8135

[he]:
- Accuracy  : 0.7570
- Precision : 0.7731
- Recall    : 0.8539
- F1-Score  : 0.8115
```

Berdasarkan hasil eksperimen, inisialisasi bobot dengan nilai nol (zeros) menunjukkan kegagalan pada proses konvergensi model. Hal ini dapat dilihat dari kurva *loss* yang langsung mendatar sejak awal *epoch*. Kegagalan ini disebabkan oleh *Symmetry Breaking Problem* yaitu ketika seluruh matriks bobot dan gradien menumpuk tepat di angka nol. Karena tidak ada pemecahan simetri, seluruh neuron menerima pembaruan gradien yang identik sehingga gagal belajar. Akibatnya, model mengambil jalan pintas dengan menebak kelas mayoritas yang ditandai dengan nilai *Recall* 1.0000 serta akurasi di angka 0.6125.

Sebaliknya, inisialisasi acak berhasil memecah simetri tersebut yang terbukti dari sebaran bobot botol dan gradien yang bervariasi serta kurva *loss* yang turun secara eksponensial. Dari grafik *learning curve*, dapat dilihat bahwa random normal memberikan pergerakan yang lebih stabil hingga akhir pembelajaran. Hal ini selaras dengan metrik akhir dari random normal yang memiliki akurasi 76,05% dan F1-Score 0.8167. Nilai ini mengungguli random uniform dengan akurasi 74,15%. Random uniform, xavier, dan he menunjukkan adanya indikasi *overfitting* di tengah training.

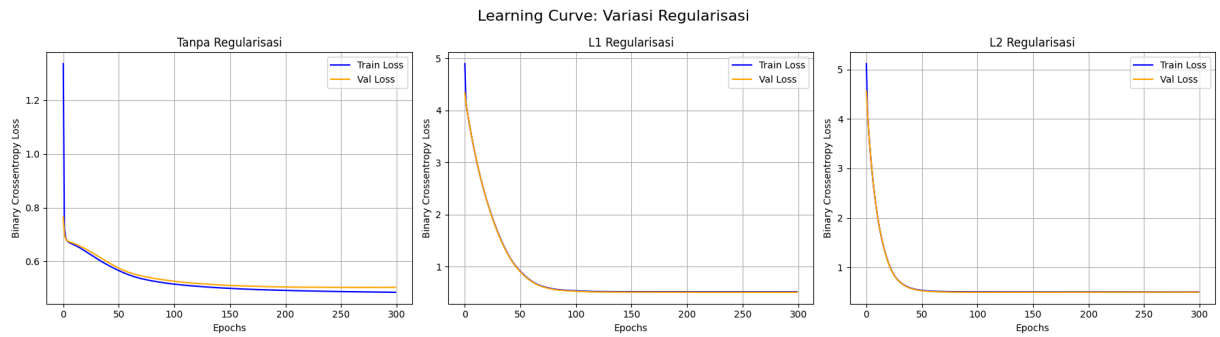
Kesimpulannya adalah inisialisasi bobot secara acak, khususnya dengan distribusi normal, merupakan syarat agar *neural network* mampu memecah simetri dan mempelajari data secara efektif.

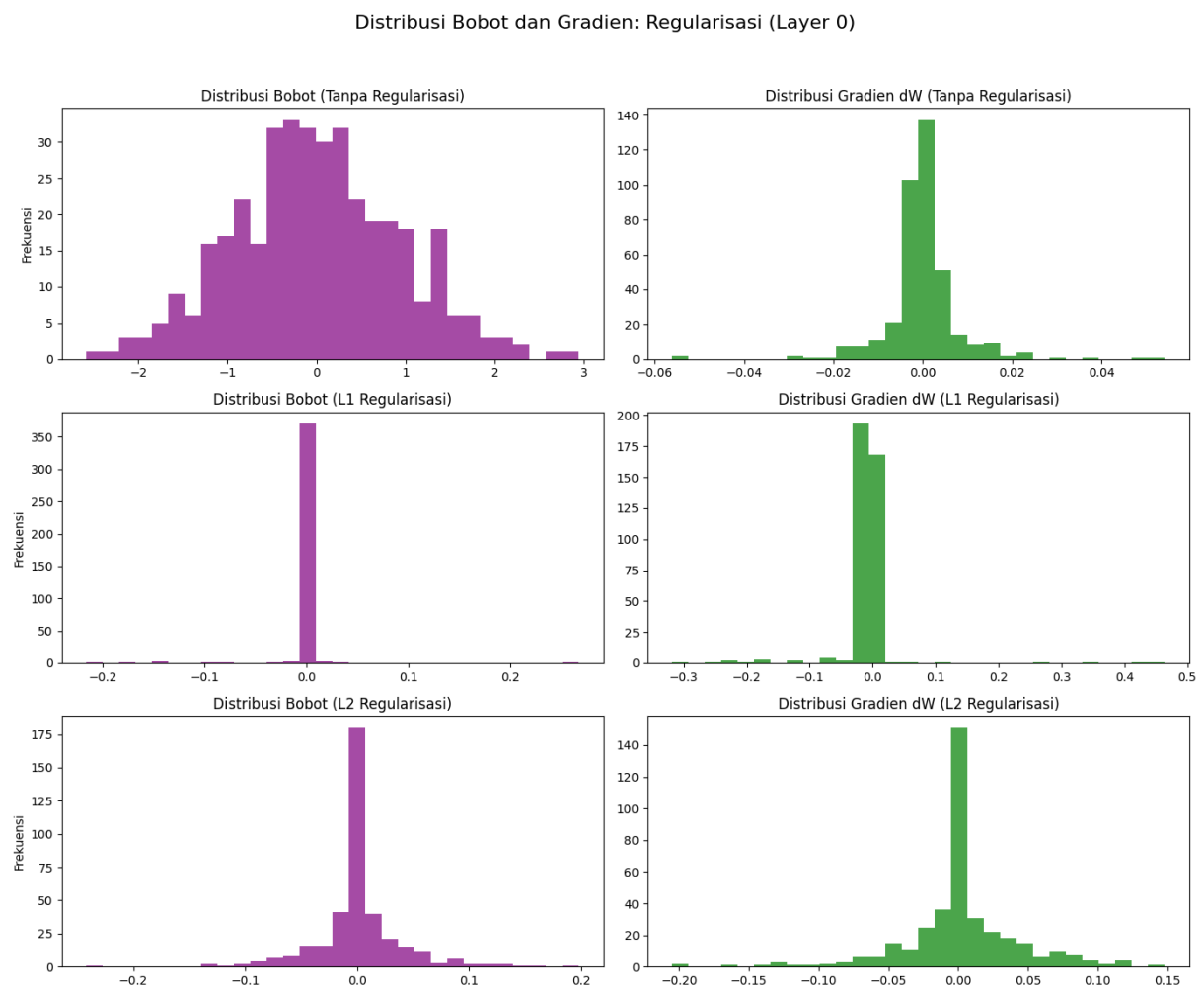
Pengaruh Regularisasi

Berikut merupakan hasil eksperimen pengaruh regularisasi.

Pengaruh Regularisasi

```
Eksperimen Pengaruh Regularisasi
Training Model: Tanpa Regularisasi
Training Model: L1 Regularisasi
Training Model: L2 Regularisasi
```






```
Evaluasi Prediksi : Regularisasi
```

```
[Tanpa Regularisasi]:
- Accuracy  : 0.7605
- Precision : 0.7687
- Recall    : 0.8710
- F1-Score  : 0.8167

[L1 Regularisasi]:
- Accuracy  : 0.7620
- Precision : 0.7585
- Recall    : 0.8971
- F1-Score  : 0.8220

[L2 Regularisasi]:
- Accuracy  : 0.7635
- Precision : 0.7629
- Recall    : 0.8906
- F1-Score  : 0.8218
```

Berdasarkan hasil eksperimen, penerapan *regularization* L1 dan L2 menyebabkan nilai *loss* awal melonjak tinggi dibandingkan tanpa *regularization.* Hal ini adalah suatu hal yang wajar karena adanya penambahan nilai penalti bobot. Meskipun nilai *loss* awalnya tinggi, kurva pembelajaran model dengan L1 atau L2 mampu turun secara eksponensial dan stabil. Hal ini juga terbukti efektif membuat grafik *validation loss* berhimpit dengan *training loss* hingga akhir *epoch* yang menandakan model dapat meredam *overfitting.*

Dampak dari penalti tersebut juga dapat dilihat pada histogram distribusi bobot. Tanpa *regularization*, matriks bobot tersebar secara bebas dan meluas pada rentang -3 hingga 3. Saat L1 diterapkan, penalti memaksa mayoritas bobot menyusut menjadi tepat nol. Sementara, penerapan L2 memberikan penalti kuadratik yang membatasi nilai ekstrim. Pengendalian distribusi bobot ini juga meningkatkan metrik performa model. Model tanpa regularisasi menghasilkan akurasi 76,05% dengan F1-Score 0,8167, model dengan L2 menghasilkan akurasi 76,35% dengan F1-Score 0,8218, dan model dengan L1 menghasilkan akurasi 76,20% dengan F1-Score 0,8220.

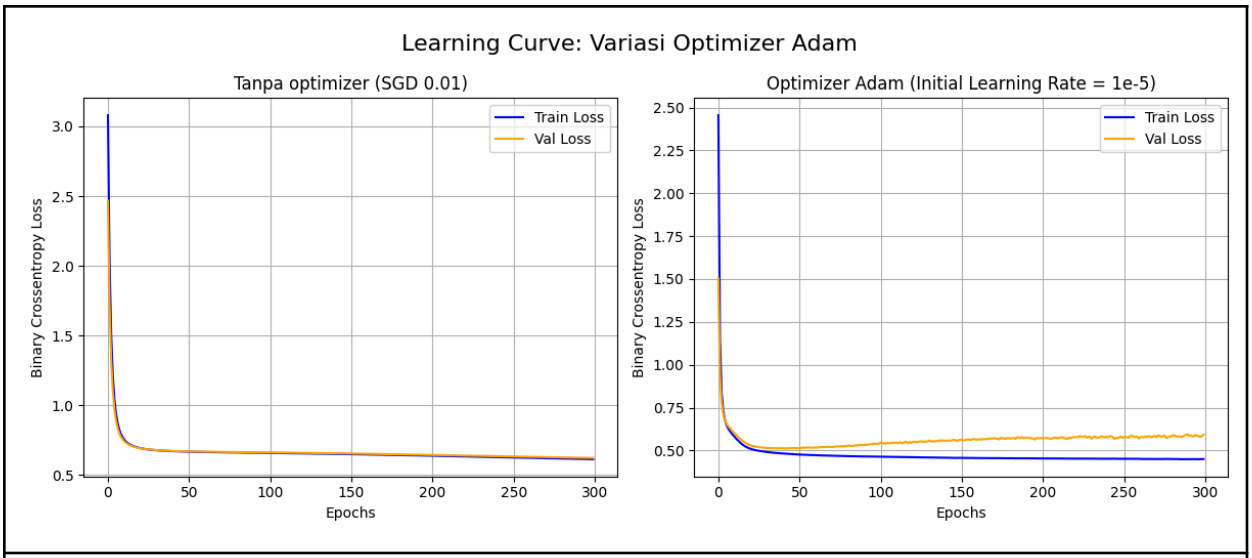
Kesimpulannya adalah penerapan regularisasi L1 maupun L2 efektif digunakan untuk membatasi variansi bobot dan meningkatkan performa dari model.

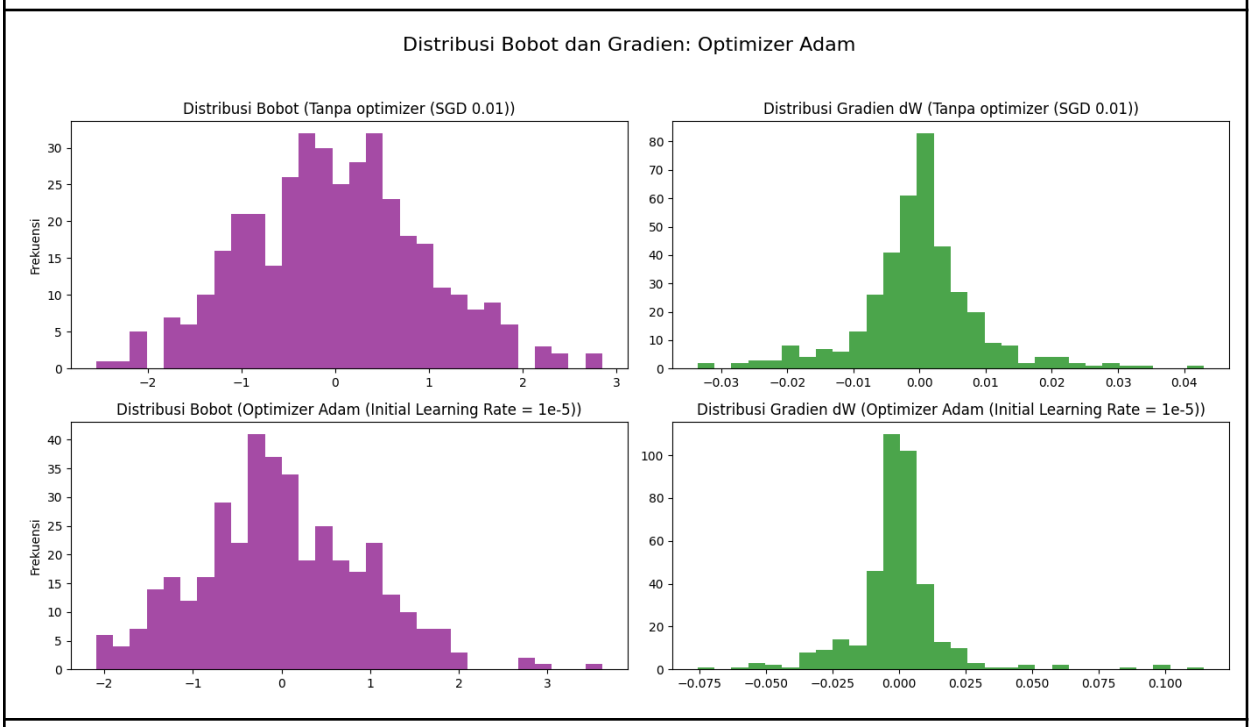
Pengaruh Optimizer

Berikut merupakan hasil eksperimen pengaruh penggunaan Optimizer Adam.

Pengaruh Optimizer

`Eksperimen Pengaruh Optimizer`
`Training Model: Tanpa Optimizer (SGD lr=0.01)`
`Training Model: Optimizer Adam (lr = 1e-5)`






```
Evaluasi Prediksi : Optimizer Adam

[Tanpa optimizer (SGD 0.01)]:
- Accuracy  : 0.6350
- Precision : 0.6495
- Recall    : 0.8776
- F1-Score  : 0.7465

[Optimizer Adam (Initial Learning Rate = 1e-5)]:
- Accuracy  : 0.7455
- Precision : 0.7668
- Recall    : 0.8400
- F1-Score  : 0.8017
```

Berdasarkan hasil eksperimen, penggunaan optimizer Adam memberikan percepatan konvergensi yang lebih agresif daripada Stochastic Gradient Descent. Pada grafik *learning curve*, *traning loss* model dengan Adam menukik sejak awal epoch. Namun, hal ini memicu adanya overfitting yang terlihat dari garis validation loss yang membelok naik. Sifat adaptif dari Adam dapat dilihat pada sebaran distribusi gradien yang lebih luas antara -0.075 hingga 0.100.

Dampak dari sebaran distribusi gradien yang adaptif sangat signifikan terhadap performa model. Model SGD hanya menghasilkan akurasi 63,50% dan F1-Score 0,7465, sedangkan dengan optimizer Adam, akurasi meningkat drastis menjadi 74,55% dan F1-Score 0,8017.

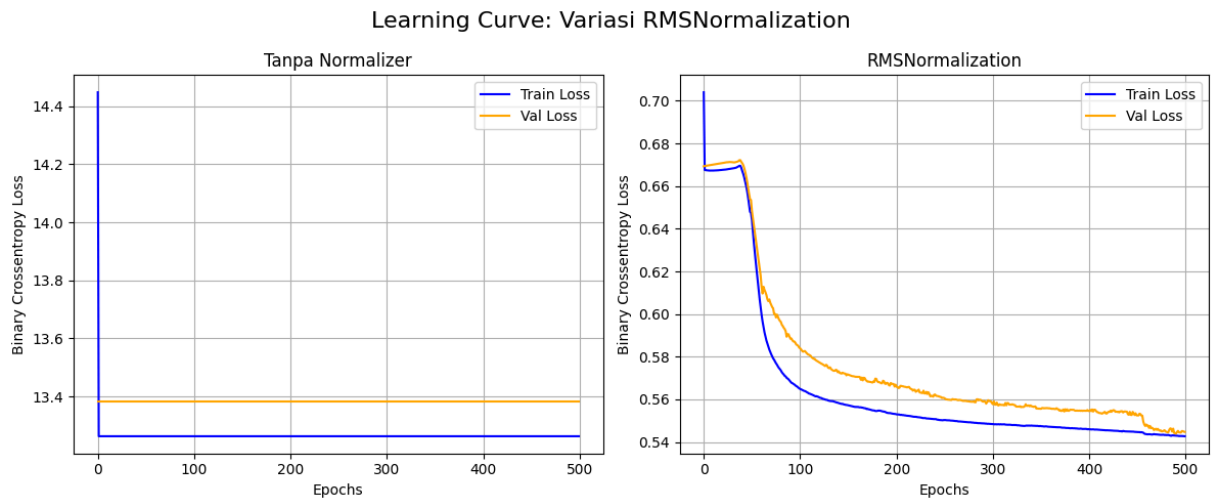
Kesimpulannya adalah optimizer Adam terbukti krusial dan efektif dalam mempercepat proses pembelajaran. Namun, pada dataset ini perlu ditambahkan dengan regularisasi agar meredam overfitting.

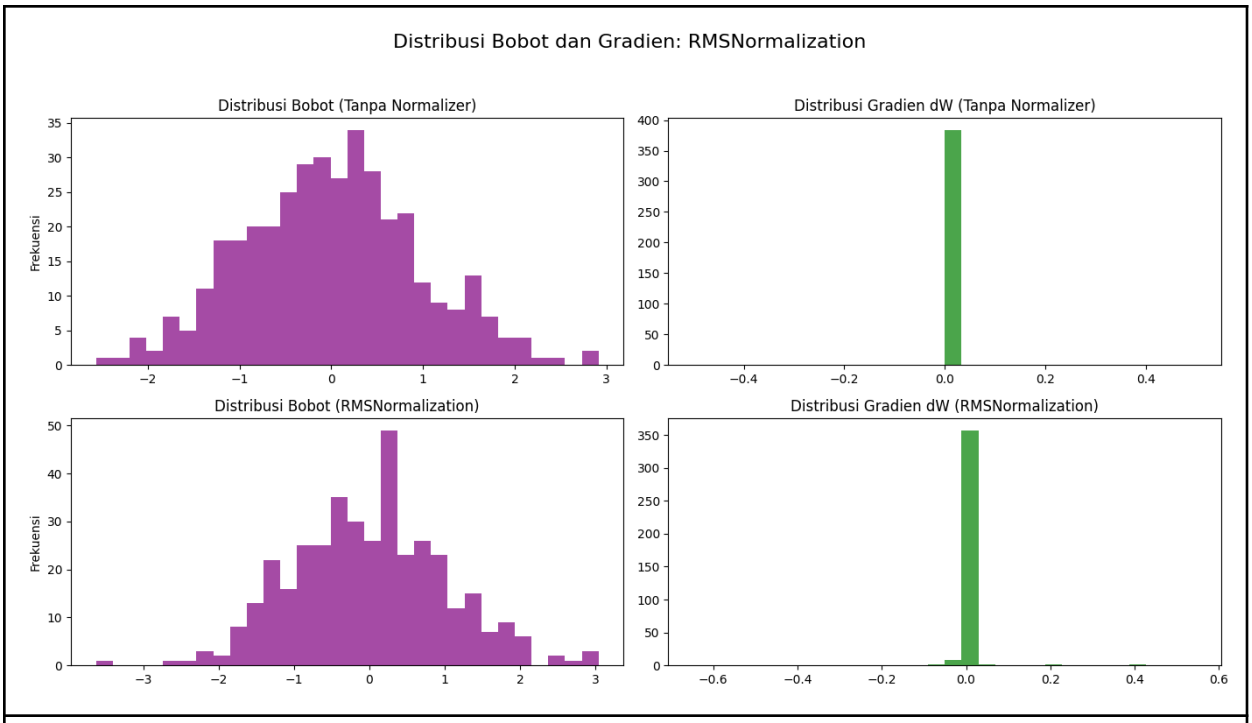
Pengaruh RMS Normalizer

Berikut merupakan hasil eksperimen RMS Normalizer.

Pengaruh RMS Normalizer

`Eksperimen Pengaruh Normalizer`
`Training Model: Tanpa Normalizer(SGD)`
`Training Model: RMSNormalizer`






```
Evaluasi Prediksi : RMSNormalization

[Tanpa Normalizer]:
- Accuracy  : 0.6125
- Precision : 0.6125
- Recall    : 1.0000
- F1-Score  : 0.7597

[RMSNormalization]:
- Accuracy  : 0.7185
- Precision : 0.7071
- Recall    : 0.9224
- F1-Score  : 0.8006
```

Berdasarkan hasil eksperimen, model tanpa adanya normalisasi menunjukkan kegagalan konvergensi. Hal ini dapat dilihat dari kurva *loss* yang langsung mendatar sejak *epoch* awal. Kegagalan ini disebabkan oleh tidak adanya pembaruan bobot (*vanishing gradient)* yang dapat dilihat pada histogram gradien yang menumpuk tepat di angka nol. Sebaliknya, penerapan RMS Normalizer dapat meneruskan propagasi. Meski sempat stagnan di *epoch* awal, *learning curve* model RMS ini dapat menukik turun mencapai titik konvergensi stabil.

Tanpa normalizer, model mengambil jalan pintas untuk menebak kelas mayoritas ditandai dengan nilai Recall 1.0000 dan akurasi 61,25%. Sedangkan, dengan RMS Normalizer angka akurasi mencapai 71,85% dan F1-Score 0,8006.

Kesimpulannya adalah penerapan RMS Normalizer terbukti efektif untuk menjaga skala variansi aktivasi antar layer sehingga mencegah *vanishing gradient* dan memastikan model dapat belajar secara optimal.

Perbandingan dengan Library Sklearn

Berikut merupakan hasil eksperimen perbandingan model FFNN From Scratch dengan Library Sklearn (MLPClassifier).

Perbandingan dengan Library Sklearn (MLPClassifier)

`Training Model FFNN From Scratch`
`Training Model Sklearn MLPClassifier`
`Perbandingan Evaluasi Prediksi`
 	
`[FFNN From Scratch]:`
`- Accuracy  : 0.7605`
`- Precision : 0.7687`
`- Recall  : 0.8710`
`- F1-Score  : 0.8167`
 	
`[Scikit-Learn (MLPClassifier)]:`
`- Accuracy  : 0.7510`
`- Precision : 0.7695`
`- Recall  : 0.8473`
`- F1-Score  : 0.8065`

Berdasarkan hasil evaluasi prediksi tersebut, model FFNN From Scratch memiliki performa yang sedikit lebih unggul dibandingkan dari Library Sklearn dengan MLPClassifier. Model sendiri menghasilkan akurasi 76,05% dan F1-Score 0,8167, sedangkan Model Sklearn memilki akurasi 75,10% dan F1-Score 0,8065.

Selisih performa ini disebabkan oleh strategi inisialisasi bobot awal yang berbeda. Sklearn menggunakan inisialisasi Glorot/xavier Uniform, sementara model sendiri menggunakan distribusi Random Normal. Pada eksperimen ini, nilai bobo acak dari model sendiri kemungkinan kebetulan berada di titik awal kurva *loss* yang menguntungkan sehingga mencapai lembah gradien yang lebih optimal.

Bab IV: Kesimpulan dan Saran

Kesimpulan

Dari seluruh hasil eksperimen, dapat disimpulkan bahwa:

- Arsitektur yang lebih dangkal tetapi lebar terbukti lebih stabil dibandingkan yang sangat dalam. Depth 2 menghasilkan akurasi terbaik (75,25%), sedangkan Depth 4 gagal dalam konvergensi akibat *vanishing gradient* yang menyebabkan model hanya menebak kelas mayoritas. Penambahan width mempercepat konvergensi tetapi tidak meningkatkan akurasi akhir secara signifikan, dengan ketiga variasi (4, 16, 64 neuron) menghasilkan akurasi yang stabil di kisaran 75-76%.
- Fungsi aktivasi linear menghasilkan performa terbaik dengan akurasi 76,85% dan F1-Score 0,8233. Hal ini berarti fitur pada dataset ini cukup mudah dipisahkan secara linear. Fungsi tanh menghasilkan performa terendah (74,40%) serta mengalami sedikit overfitting. Fungsi aktivasi ReLU juga dapat menjadi alternatif.
- Learning rate sedang (lr=0,01) memberikan hasil terbaik dengan akurasi 76,05% dan F1-Score 0,8167. Learning rate terlalu besar (lr=0,5) menyebabkan fluktuasi pada validation loss dan sulit konvergen, sedangkan learning rate terlalu kecil (lr=0,0001) menyebabkan model terjebak di minimum lokal dengan akurasi hanya 60,25%.
- Inisialisasi bobot dengan nilai nol (*zeros*) sangat buruk akibat symmetry breaking problem, di mana seluruh neuron menerima gradien identik sehingga tidak bisa belajar. Inisialisasi random normal menghasilkan performa terbaik (akurasi 76,05%, F1-Score 0,8167) dan kurva pembelajaran paling stabil dibandingkan metode lainnya.
- Penggunaan regularisasi L1 dan L2 cukup efektif untuk membatasi variansi bobot dan mengurangi overfitting, terbukti dari validation loss yang berhimpit dengan training loss hingga akhir epoch. Keduanya meningkatkan performa dibanding tanpa regularisasi, dengan L1 mencapai F1-Score 0,8220 dan L2 mencapai F1-Score 0,8218.
- Optimizer Adam mempercepat konvergensi secara signifikan dengan akurasi 74,55% dan F1-Score 0,8017, lebih unggul dibanding SGD murni yang hanya menghasilkan akurasi 63,50% pada eksperimen yang sama. Namun, Adam memicu overfitting yang terlihat dari validation loss yang membelok naik.
- RMS Normalization terbukti efektif mencegah *vanishing gradient*. Tanpa normalisasi, model gagal konvergen dan hanya menebak kelas mayoritas (akurasi 61,25%), sedangkan dengan RMS Normalization model mampu mencapai akurasi 71,85% dan F1-Score 0,8006.
- Implementasi FFNN *from scratch* menyamai bahkan sedikit melampaui performa sklearn MLPClassifier, dengan akurasi 76,05% vs 75,10% dan F1-Score 0,8167 vs 0,8065, membuktikan kebenaran implementasi forward dan backward propagation yang dibangun.

Saran

- Untuk arsitektur jaringan, depth 2 dengan lebar yang cukup sudah terbukti optimal untuk dataset ini. Jika ingin menambah depth pada dataset yang lebih kompleks, perlu dikombinasikan dengan teknik seperti RMS Normalization atau residual connection untuk mengatasi vanishing gradient.

- Untuk fungsi aktivasi, linear dan sigmoid sudah memberikan hasil terbaik pada dataset ini. Pada dataset yang lebih kompleks dan tidak linearly separable, mungkin akan lebih baik menggunakan variasi ReLU seperti leaky_relu atau swish yang juga terbukti stabil pada eksperimen ini.
- Untuk learning rate, nilai 0,01 sudah cukup optimal untuk SGD. Kedepannya dapat dipertimbangkan penggunaan learning rate scheduler agar learning rate dapat menyesuaikan secara adaptif selama training, mengurangi ketidakstabilan yang terjadi pada learning rate besar.
- Untuk optimizer, Adam sangat direkomendasikan karena memberikan percepatan konvergensi yang signifikan. Namun perlu dikombinasikan dengan regularisasi L1 atau L2 untuk mengurangi overfitting yang ditimbulkan, karena kedua teknik tersebut terbukti meningkatkan performa model secara bersamaan pada eksperimen ini.
- Untuk inisialisasi bobot, random normal sudah memberikan hasil terbaik dan stabil. Inisialisasi zeros harus dihindari karena terbukti menyebabkan kegagalan total akibat symmetry breaking problem. Xavier dan He dapat menjadi alternatif, walau perlu hati-hati karena memungkinkan overfitting di tengah training.

Pembagian Tugas

Nama	NIM	Tugas
Muhammad Dicky Isra	13523075	Desain sistem, Forward Propagation, Bonus initialization, Laporan
Guntara Hambali	13523114	Backward Propagation, Optimizer, RMSNorm, Laporan
Reza Ahmad Syarif	13523119	Utility Functions for Backward Propagation (Loss, Loss derivative, Activation, Activation derivative, L1, L2), Notebook Eksperimen, Laporan

Lampiran

Repositori Github: DaDecky/ml-tubes-1
Form penggunaan AI:

- Muhammad Dicky Isra/13523075
  Form Penggunaan AI_Muhammad Dicky Isra_13523075
- Guntara Hambali / 13523114
  Form Penggunaan AI_Guntara Hambali_13523114
- Reza Ahmad Syarif / 13523119
  Form Penggunaan AI_Reza Ahmad Syarif_13523119

Daftar Pustaka

https://www.geeksforgeeks.org/deep-learning/xavier-initialization/

https://www.geeksforgeeks.org/deep-learning/kaiming-initialization-in-deep-learning/

https://prateekvishnu.medium.com/xavier-and-he-normal-he-et-al-initialization-8e3d7a087528

https://jvgd.medium.com/rms-norm-acc968eeac1f

https://keras.io/api/layers/normalization_layers/rms_normalization/

https://www.geeksforgeeks.org/deep-learning/adam-optimizer/

https://keras.io/api/optimizers/adam/

The Math of Loss Functions - Brandon Da Silva

Mean Squared Error in Python - GeeksforGeeks

Binary Cross Entropy/Log Loss for Binary Classification - GeeksforGeeks

12 Types of Activation Functions in Neural Networks: A Comprehensive Guide | by Sushmita Nandi | Medium

Swish Activation Function - GeeksforGeeks

Swish: A new activation function from Google Brain | by Rittika Sur | Medium

Leaky Relu Activation Function in Deep Learning - GeeksforGeeks

Activation functions: ReLU vs. Leaky ReLU | by Srikari Rallabandi | Medium